Python OOP: Polymorphism & Abstraction

Focused practice problem set covering Dynamic Polymorphism, Duck Typing, and Abstract Base Classes (abc module).

5 POLYMORPHISM TASKS | 5 ABSTRACTION TASKS

Task 1: Shape Area Calculator

Polymorphism

PROBLEM STATEMENT

Design a shape area calculation framework using method overriding:

1. Create a base class `Shape` with a method `calculate_area()` that returns `0.0`.
2. Create a subclass `Circle` that initializes with `radius` and overrides `calculate_area()` to return `3.14159 * radius`^{2}.
3. Create a subclass `Rectangle` that initializes with `width` and `height` and overrides `calculate_area()` to return `width * height`.

INPUT FORMAT

```
Circle(radius=5.0)
Rectangle(width=4.0, height=6.0) Actions: -
circle.calculate_area() -
rectangle.calculate_area()
```

SAMPLE OUTPUT

```
78.53975 24.0
```

Task 2: Payment Processor Gateway

Polymorphism / Duck Typing

PROBLEM STATEMENT

Design a payment dispatching system utilizing duck typing:

1. Create a class `CreditCardPayment` with a method `process_payment(amount)` that returns `"Processing credit card payment of $[amount]"`.
2. Create a class `PayPalPayment` with a method `process_payment(amount)` that returns `"Processing PayPal payment of $[amount]"`.
3. Create a standalone function `process_transaction(processor, amount)` that accepts any payment processor object and calls its `process_payment()` method.

INPUT FORMAT

```
Processors: CreditCardPayment,
PayPalPayment Actions: -
process_transaction(cc, 150.0) -
process_transaction(pp, 85.5)
```

SAMPLE OUTPUT

```
Processing credit card payment of
$150.0 Processing PayPal payment of $85.5
```

Task 3: Notification Dispatcher

Polymorphism

PROBLEM STATEMENT

Design a polymorphic notification engine:

1. Create three distinct classes: `EmailNotification`, `SMSNotification`, and `PushNotification`.
2. Each class must implement a `send(message)` method returning:
   - Email: `"Email sent: [message]"`
   - SMS: `"SMS sent: [message]"`
   - Push: `"Push Notification sent: [message]"`
3. Iterate over a list containing instances of all three objects and call `send()` on each.

INPUT FORMAT

```
Message: "System Update Scheduled"
```

SAMPLE OUTPUT

```
Email sent: System Update Scheduled
```

Task 4: Media Player Controls

Polymorphism

PROBLEM STATEMENT

Design a uniform interface for media playback devices:

1. Create a class `AudioPlayer` with methods `play()` returning `"Playing audio track"` and `stop()` returning `"Audio stopped"`.
2. Create a class `VideoPlayer` with methods `play()` returning `"Playing video stream"` and `stop()` returning `"Video stopped"`.
3. Implement a function `start_and_stop_media(player)` that calls `play()` followed by `stop()` on any passed player object.

INPUT FORMAT

```
Players: AudioPlayer, VideoPlayer
Actions: - start_and_stop_media(audio) -
start_and_stop_media(video)
```

SAMPLE OUTPUT

```
Playing audio track Audio stopped
Playing video stream Video stopped
```

Task 5: Tax Calculator for Different Products

Polymorphism

PROBLEM STATEMENT

Design a product tax computation system using a polymorphic interface:

1. Create a class `StandardProduct` that initializes with `price` and implements `calculate_tax()` returning 5% tax (`price * 0.05`).
2. Create a class `LuxuryProduct` that initializes with `price` and implements `calculate_tax()` returning 20% tax (`price * 0.20`).
3. Implement a function `get_total_tax(products_list)` that iterates over mixed product objects and computes the sum of all individual taxes.

INPUT FORMAT

```
Products: StandardProduct(100.0),
LuxuryProduct(500.0) Actions: -
get_total_tax([std, lux])
```

SAMPLE OUTPUT

```
105.0
```

Task 6: Abstract Bank Account

Abstraction (abc)

PROBLEM STATEMENT

Enforce a bank account contract using Python's `abc` module:

1. Create an abstract base class `BankAccount` inheriting from ABC. Define abstract methods `deposit(amount)` and `withdraw(amount)` decorated with `@abstractmethod`.
2. Create a concrete subclass `CheckingAccount` initializing with `balance` that implements both abstract methods:
   - `deposit(amount)`: adds to balance and returns updated balance.
   - `withdraw(amount)`: deducts from balance if sufficient funds exist, else returns `"Insufficient funds"`.

INPUT FORMAT

```
CheckingAccount(balance=200.0)
Actions: - deposit(50.0) - withdraw(300.0) -
withdraw(100.0)
```

SAMPLE OUTPUT

```
250.0 Insufficient funds 150.0
```

Task 7: Abstract Database Connector

Abstraction (abc)

PROBLEM STATEMENT

Define an abstract database interface using ABC and `@abstractmethod`:

1. Create an abstract class `DatabaseConnector` with abstract methods `connect()` and `execute_query(query)`.
2. Create a concrete class `MySQLConnector` initializing with db_name that implements both abstract methods:
   - `connect()`: returns `"Connected to MySQL database: [db_name]"`
   - `execute_query(query)`: returns `"Executing '[query]' on [db_name]"`

INPUT FORMAT

```
DB Name: "production_db" Actions: - connect() - execute_query("SELECT * FROM users")
```

SAMPLE OUTPUT

```
Connected to MySQL database: production_db Executing 'SELECT * FROM users' on production_db
```

Task 8: Abstract Appliance Power Switch

Abstraction (abc)

PROBLEM STATEMENT

Design a hardware interface specification using abstraction:

1. Create an abstract base class `SwitchableDevice` using ABC with abstract methods `turn_on()` and `turn_off()`.
2. Create a concrete class `SmartBulb` that initializes with `device_name` and tracks state attribute `is_on` (boolean, default `False`).
3. Implement `turn_on()` setting `is_on = True` and returning `"[device_name] light turned ON"`, and `turn_off()` setting `is_on = False` and returning `"[device_name] light turned OFF"`.

INPUT FORMAT

```
Device Name: "Bedroom Bulb" Actions: - turn_on() - turn_off()
```

SAMPLE OUTPUT

```
Bedroom Bulb light turned ON Bedroom Bulb light turned OFF
```

Task 9: Abstract Data Parser

Abstraction (abc)

PROBLEM STATEMENT

Design an extensible data parsing architecture using abstract base classes:

1. Create an abstract class `DataParser` with abstract method `parse_data(raw_data)`.
2. Create concrete subclass `JSONParser` implementing `parse_data()` to return `"Parsed JSON dictionary from: [raw_data]"`.
3. Create concrete subclass `CSVParser` implementing `parse_data()` to return `"Parsed CSV rows from: [raw_data]"`.

INPUT FORMAT

```
JSON string: '{"key": "val"}' CSV string: "a,b,c" Actions: - json_parser.parse_data(json_str) - csv_parser.parse_data(csv_str)
```

SAMPLE OUTPUT

```
Parsed JSON dictionary from: {"key": "val"} Parsed CSV rows from: a,b,c
```

Task 10: Abstract Employee Bonus Engine

Abstraction (abc)

PROBLEM STATEMENT

Design a flexible bonus calculation engine using abstract classes:

1. Create an abstract base class `BonusCalculator` inheriting from ABC with an abstract method `calculate_bonus(base_salary)`.
2. Create concrete subclass `SalesBonus` implementing `calculate_bonus()` returning 15% of salary (`base_salary * 0.15`) plus a flat $500.0 commission.
3. Create concrete subclass `TechBonus` implementing `calculate_bonus()` returning 10% of salary (`base_salary * 0.10`) plus a flat $1000.0 tech stipend.

INPUT FORMAT

```
Base Salary: 60000.0 Actions: - sales_bonus.calculate_bonus(60000.0) - tech_bonus.calculate_bonus(60000.0)
```

SAMPLE OUTPUT

```
9500.0 7000.0
```

Core OOP Rules & Guidelines

- **Polymorphism (Tasks 1–5):** Rely on dynamic dispatch or duck typing. Child classes override parent methods or adhere to standard interface method signatures without requiring abstract class enforcement.
- **Abstraction (Tasks 6–10):** Use `from abc import ABC, abstractmethod`. Abstract classes CANNOT be instantiated directly; concrete subclasses MUST implement all `@abstractmethod` decorated routines to be instantiated.
- **Clean Signatures:** Ensure method names and argument signatures match problem specifications exactly across all subclass implementations.